

UNIVERSIDADE DA BEIRA INTERIOR
Covilhã | Portugal

Relatório do Trabalho Prático 2 - Gestão de Farmácia

Afonso Esteves - 55047
Felipe Dourado - 53394
Tiago Pereira - 55019

12 de novembro de 2025

Conteúdo

Resumo **3**

1 Introdução **4**
1.1 Âmbito, enquadramento e motivação 4

2 Contextualização do Projeto **4**
2.1 Identificação dos Desenvolvedores 4
2.1.1 Desenvolvedores Envolvidos 4
2.2 Contextualização Académica 4

3 Especificações Técnicas **5**
3.1 Ambiente de Desenvolvimento 5
3.2 Tecnologias e Bibliotecas 5
3.3 Práticas de Engenharia 5

4 Utilização **6**
4.1 Requisitos Prévios 6
4.2 Compilação e Execução 6
4.3 Execução de Testes Unitários 6
4.4 Geração de Documentação JavaDoc 7

5 Estrutura da Solução Construída **7**
5.1 Visão Geral 7
5.2 Detalhamento dos Ficheiros 7

6 Testes e Documentação **8**

7 Funcionalidades Implementadas **9**

8 Implementação de Funcionalidades **9**
8.1 Classe FarmaciaGUI 9
8.2 Classe GestorDados 10
8.3 Classe FarmaciaTest 11

9 Considerações de Implementação **12**

10 Conclusão **12**

11 Referências Bibliográficas **14**

Resumo

Este relatório documenta a evolução de um sistema de gestão de farmácia desenvolvido no âmbito da disciplina de Programação Orientada a Objetos. A segunda entrega introduziu uma interface gráfica baseada em Java Swing, persistência de dados através de serialização de objetos, testes unitários JUnit 5 e documentação com JavaDoc. O código foi reorganizado em pacotes (`app`, `modelo`, `gui`, `persistencia`, `test`) e beneficiou de refatoração orientada por princípios de POO, resultando numa aplicação mais robusta, reutilizável e preparada para futuras extensões.

Palavras-chave e *Keywords*

Java; Swing; GUI; Programação Orientada a Objetos; Persistência de Dados; Serialização; JUnit; Javadoc.

1 Introdução

1.1 Âmbito, enquadramento e motivação

Este documento descreve o Trabalho Prático 2 (TP2) da disciplina, que dá continuidade ao sistema originalmente desenvolvido no TP1. O objetivo é consolidar competências de POO ao migrar uma aplicação de consola para uma interface gráfica, garantindo persistência de dados e qualidade através de testes e documentação formal. A motivação passa por aproximar o projeto de cenários reais: uma interface amigável, dados duráveis, testes automatizados e documentação técnica.

2 Contextualização do Projeto

2.1 Identificação dos Desenvolvedores

Os autores são estudantes do curso de Informática Web, Móvel e na Nuvem da Universidade da Beira Interior (UBI), inscritos na unidade curricular de Programação Orientada a Objetos.

2.1.1 Desenvolvedores Envolvidos

- **Afonso Esteves (55047)**
- **Felipe Dourado (53394)**
- **Tiago Pereira (55019)**

2.2 Contextualização Académica

O projeto foi desenvolvido com os seguintes objetivos principais:

- Desenvolver GUIs com Java Swing e programação orientada a eventos.
- Aplicar herança, interfaces, sobrecarga e sobreposição.
- Implementar persistência de dados com serialização de objetos.
- Criar testes unitários com JUnit 5.
- Gerar documentação JavaDoc.
- Produzir documentação escrita consolidando as decisões de design.

3 Especificações Técnicas

3.1 Ambiente de Desenvolvimento

- **Sistema Operativo**: macOS 15 (Darwin 25.0.0).
- **Shell**: `/bin/zsh`, utilizado para compilação, execução, testes e geração de documentação.
- **IDE / Editores**: IntelliJ IDEA e Cursor, suportando funcionalidades de refatoração, inspeções e integração com VCS.
- **Controlo de Versões**: Git (repositório local e integração com plataformas remotas).

3.2 Tecnologias e Bibliotecas

- **Linguagem de Programação**: Java 21, permitindo explorar recursos modernos da linguagem mantendo compatibilidade retroativa.
- **Biblioteca Gráfica**: Java Swing e AWT para construção da interface (componentes como `JFrame`, `JPanel`, `JTable`, `JDialog`).
- **Persistência**: Serialização nativa com `ObjectOutputStream`/`ObjectInputStream`, garantindo persistência sem recurso a bases de dados relacionais ou NoSQL.
- **Testes**: Framework JUnit 5, distribuída através do artefacto `junit-platform-console-sta` permitindo execução em linha de comandos e integração CI.
- **Documentação**: JavaDoc para produção de documentação HTML da API.
- **Bibliotecas Standard**: `java.util` (coleções, `Optional`), `java.io` (streams de objetos) e `java.awt.event` (listeners).

3.3 Práticas de Engenharia

- **Arquitetura em Camadas**: separação clara entre domínio, vista e persistência.
- **POO Avançada**: utilização de herança (`Pessoa` →`Cliente`), abstrações (`Vendavel`), sobrecarga/sobrescrita (`Encomenda`), encapsulamento e composição (`Cliente`–`Morada`).

- **Programação Orientada a Eventos**: modelo de listeners para captar interações na GUI.
- **Testes Automatizados**: validação da lógica de negócio com JUnit e execução regular durante o desenvolvimento.
- **Documentação Contínua**: manutenção do JavaDoc e do relatório técnico (`CONTEXTO.md`) alinhados com o estado do código.

4 Utilização

4.1 Requisitos Prévios

- **Java Development Kit (JDK) 17 ou superior**: necessário para compilar e executar.
- **Acesso à linha de comandos**: Terminal (macOS) ou CMD/PowerShell (Windows).

4.2 Compilação e Execução

1. Navegar até ao diretório raiz do projeto (`TP-02/`).
2. Compilar código de produção:

```
javac -cp "lib/junit-platform-console-standalone-1.10.2.jar:src" -d out \
  $(find src -name "*.java" ! -path "*/test/*")
```

3. Compilar classes de teste:

```
javac -cp "lib/junit-platform-console-standalone-1.10.2.jar:out" -d out \
  $(find src/test -name "*.java")
```

4. Executar a aplicação GUI:

```
java -cp "out" app.Main
```

4.3 Execução de Testes Unitários

1. Após a compilação, executar:

```
java -jar lib/junit-platform-console-standalone-1.10.2.jar \
  --class-path out --scan-class-path
```

4.4 Geração de Documentação JavaDoc

1. Na raiz do projeto:

   ```
   javadoc -d doc -sourcepath src app modelo gui persistencia
   ```

2. O resultado fica em `doc/index.html`.

5 Estrutura da Solução Construída

5.1 Visão Geral

O código encontra-se organizado por responsabilidade:

- **Pacote** `app`: ponto de entrada `Main`.
- **Pacote** `modelo`: domínio da aplicação (entidades e regras).
- **Pacote** `gui`: interface Swing (janela principal e painéis).
- **Pacote** `persistencia`: gestão de ficheiros para serialização.
- **Pacote** `test`: testes JUnit 5.

5.2 Detalhamento dos Ficheiros

- `src/app/Main.java`: Carrega dados, inicializa `Farmacia` e arranca a GUI. Lança a aplicação na Event Dispatch Thread.
- `src/modelo/Pessoa.java`: Classe abstrata base com nome e NIF. Obriga subclasses a implementar `toString()`.
- `src/modelo/Cliente.java`: Extende `Pessoa`, adiciona `Morada` e serialização. Representa clientes da farmácia.
- `src/modelo/Morada.java`: Value object com rua, cidade e código postal.
- `src/modelo/Vendavel.java`: Interface com `getNome()` e `getPreco()`.
- `src/modelo/Produto.java`: Implementa `Vendavel`; gere ID, preço e stock, com métodos de validação.
- `src/modelo/Encomenda.java`: Relaciona cliente e produtos, controla estados, calcula preço total, oferece construtores sobrecarregados.

- `src/modelo/Farmacia.java`: Núcleo da lógica; gere listas, valida operações, ajusta stock e estados.
- `src/persistencia/GestorDados.java`: Serializa/desserializa dados para `farmacia.dat`; atualiza contadores de IDs.
- `src/gui/FarmaciaGUI.java`: `JFrame` com `JTabbedPane`; trata encerramento com opção de guardar.
- `src/gui/PainelGestaoClientes.java`: Tabela e formulários para CRUD de clientes, com validação de NIF (9 dígitos) e código postal (0000-000), permitindo correção de NIF apenas neste contexto.
- `src/gui/PainelGestaoProdutos.java`: Gestão de produtos com validação de stock/preço e atualização da tabela.
- `src/gui/PainelGestaoEncomendas.java`: Gestão completa de encomendas (seleção de cliente, seleção/pesquisa de produtos, certificação de stock, atualização de estado e sincronização com painel de produtos), com diálogo de gestão que oferece ações de adicionar, remover e guardar alterações.
- `src/test/FarmaciaTest.java`: Testes unitários que asseguram regras de negócio (clientes, encomendas, stock).
- `lib/junit-platform-console-standalone-1.10.2.jar`: Dependência JUnit 5.
- `doc/`: JavaDoc gerado.
- `farmacia.dat`: Ficheiro de dados serializados (não versionado).
- `CONTEXTO.md`: Documento de referência técnica.

6 Testes e Documentação

- **Testes Unitários**: `FarmaciaTest` cobre operações críticas (clientes, encomendas, stock). Execução via JUnit console.
- **JavaDoc**: Geração automática para todos os pacotes, facilitando consulta da API.

7 Funcionalidades Implementadas

- Interface gráfica com três separadores: Clientes, Produtos e Encomendas.
- CRUD completo para clientes e produtos, com validação de NIF (9 dígitos) e código postal (0000-000).
- Registo, edição (incluindo atualização de stock/estado) e remoção de encomendas com sincronização imediata do stock.
- Diálogo dedicado ao ajuste de produtos da encomenda, com ações de adicionar, remover e guardar alterações.
- Persistência automática ao terminar a sessão (com confirmação).
- Validações de dados (NIF único, formato de código postal, stock suficiente, preço/stock não negativos).
- Testes unitários para garantir a consistência da lógica.

8 Implementação de Funcionalidades

8.1 Classe FarmaciaGUI

```
1  public class FarmaciaGUI extends JFrame {
2      public FarmaciaGUI(Farmacia farmacia) {
3          setTitle("Gest o de Farm cia");
4          setSize(800, 600);
5          setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
6          setLocationRelativeTo(null);
7
8          addWindowListener(new WindowAdapter() {
9              @Override
10             public void windowClosing(WindowEvent e) {
11                 int confirm = JOptionPane.showConfirmDialog(
12                     null,
13                     "Deseja guardar os dados antes de sair?",
14                     "Sair",
15                     JOptionPane.YES_NO_CANCEL_OPTION
16                 );
17                 if (confirm == JOptionPane.YES_OPTION) {
18                     GestorDados.guardarDados(
```

```
                        farmacia.getClientes(),
                        farmacia.getProdutos(),
                        farmacia.getEncomendas()
                    );
                    dispose();
                    System.exit(0);
                } else if (confirm == JOptionPane.NO_OPTION) {
                    dispose();
                    System.exit(0);
                }
            }
        });

        JTabbedPane tabbedPane = new JTabbedPane();
        PainelGestaoClientes painelClientes = new
            PainelGestaoClientes(farmacia);
        PainelGestaoProdutos painelProdutos = new
            PainelGestaoProdutos(farmacia);
        PainelGestaoEncomendas painelEncomendas =
            new PainelGestaoEncomendas(farmacia, painelProdutos);

        tabbedPane.addTab("Clientes", painelClientes);
        tabbedPane.addTab("Produtos", painelProdutos);
        tabbedPane.addTab("Encomendas", painelEncomendas);
        add(tabbedPane);
    }
}
```

Listing 1: Configuração da janela principal em FarmaciaGUI.java

8.2 Classe GestorDados

```
public static void guardarDados(
        List<Cliente> clientes,
        List<Produto> produtos,
        List<Encomenda> encomendas) {
    try (ObjectOutputStream oos =
             new ObjectOutputStream(new
                 FileOutputStream(NOME_FICHEIRO))) {
        oos.writeObject(clientes);
        oos.writeObject(produtos);
        oos.writeObject(encomendas);
    } catch (IOException e) {
```

```
11          System.err.println("Erro ao guardar os dados: " +
                e.getMessage());
12      }
13  }
14
15  @SuppressWarnings("unchecked")
16  public static Object[] carregarDados() {
17      File f = new File(NOME_FICHEIRO);
18      if (!f.exists()) {
19          return null;
20      }
21      try (ObjectInputStream ois =
22               new ObjectInputStream(new
                     FileInputStream(NOME_FICHEIRO))) {
23          List<Cliente> clientes = (List<Cliente>) ois.readObject();
24          List<Produto> produtos = (List<Produto>) ois.readObject();
25          List<Encomenda> encomendas = (List<Encomenda>)
                ois.readObject();
26
27          if (!produtos.isEmpty()) {
28              int maxIdProduto = produtos.stream()
29                      .mapToInt(Produto::getId).max().orElse(0);
30              Produto.setProximoId(maxIdProduto + 1);
31          }
32          if (!encomendas.isEmpty()) {
33              int maxIdEncomenda = encomendas.stream()
34                      .mapToInt(Encomenda::getId).max().orElse(0);
35              Encomenda.setProximoId(maxIdEncomenda + 1);
36          }
37          return new Object[]{clientes, produtos, encomendas};
38      } catch (IOException | ClassNotFoundException e) {
39          System.err.println("Erro ao carregar os dados: " +
                e.getMessage());
40          return null;
41      }
42  }
```

Listing 2: Persistência em GestorDados.java

8.3 Classe FarmaciaTest

```
1  @Test
2  void testAtualizarProdutosEncomenda() {
```

```
3       Map<Produto, Integer> produtosEncomendaOriginal = new
            HashMap<>();
4       produtosEncomendaOriginal.put(produto1, 10);
5       farmacia.registarEncomenda(cliente, produtosEncomendaOriginal);
6
7       int idEncomenda = farmacia.getEncomendas().get(0).getId();
8       Map<Produto, Integer> produtosEncomendaNova = new HashMap<>();
9       produtosEncomendaNova.put(produto2, 15);
10
11      farmacia.atualizarProdutosEncomenda(idEncomenda,
            produtosEncomendaNova);
12
13      Encomenda encomendaAtualizada =
14          farmacia.acharEncomendaPorId(idEncomenda).get();
15      assertTrue(encomendaAtualizada.getProdutos().containsKey(produto2));
16      assertEquals(100, produto1.getStock());
17      assertEquals(35, produto2.getStock());
18  }
```

Listing 3: Exemplo de teste em FarmaciaTest.java

9 Considerações de Implementação

- Utilização de encapsulamento e validações para garantir consistência (e.g., stock não negativo).
- Regras de formato para NIF (9 dígitos) e código postal (0000-000), com correção restringida ao painel de clientes.
- Dedução e reposição automáticas de stock ao registar/remover encomendas.
- Atualização de identificadores estáticos após carregamento da persistência para evitar colisões.
- Sincronização entre os painéis de encomendas e produtos para refletir alterações de stock imediatamente.

10 Conclusão

O TP2 alcançou os objetivos propostos: interface gráfica intuitiva, persistência de dados fiável, testes automatizados e documentação técnica. A

organização em pacotes torna o projeto escalável e facilita futuras evoluções, como a introdução de relatórios estatísticos ou novas interfaces. O conhecimento consolidado neste trabalho forma uma base sólida para desafios mais complexos em POO.

11 Referências Bibliográficas

- Schildt, Herbert. *Java: A Beginner's Guide*. 6th ed., McGraw-Hill Education, 2014.
- Olsson, Mikael. *Java Quick Syntax Reference*. Apress, 2014.
- Roberts, Eric S. *The Art and Science of Java*. Addison-Wesley, 2008.
- Documentação oficial do JUnit 5: `https://junit.org/junit5/docs/current/user-guide/`

Acrónimos

UBI Universidade da Beira Interior

GUI Graphical User Interface